

Billing Portal — Laravel Full Setup & Flow Documentation

1. Project Creation (Laravel)

Command to create project:

```
composer create-project laravel/laravel billing-portal  
cd billing-portal
```

Install Breeze Authentication:

```
composer require laravel/breeze --dev  
php artisan breeze:install  
npm install  
npm run build  
php artisan migrate
```

Breeze provides login, register, logout, dashboard, password hashing and auth middleware.

2. GitHub and Hostinger Deployment

Initialize Git repository and push to GitHub:

```
git init  
git add .  
git commit -m "Initial Laravel setup"  
git push -u origin main
```

Hostinger is connected to GitHub and auto-deploys on every push.

3. Hosting Folder Structure

For security, only public_html must be public.

Laravel App Location: /home/USER/laravel_app

Public Web Root: /home/USER/public_html

public_html/index.php loads Laravel from laravel_app folder.

4. Tailwind CSS and Vite Build

Local Development:

```
npm run dev
```

Production Build (Required before deploy):

```
npm run build
```

Always commit public/build folder to GitHub for Hostinger deployment.

5. Branding with Tailwind

Custom brand color added in tailwind.config.js:

brand: #1f5abc

After config change, always run npm run build and redeploy.

6. Authentication Flow

POST /login → AuthenticatedSessionController@store

Steps: Validate credentials → DB check → session regeneration → redirect to dashboard

Passwords are stored as bcrypt hashes, never plain text.

7. Database Setup

Local development uses SQLite.

Production (Hostinger) uses MySQL.

Tables are created using migrations, not models.

8. Migrations

Create migration:

php artisan make:migration create_websites_table

Run migration locally:

php artisan migrate

Run migration on Hostinger:

php artisan migrate --force

9. Models and Eloquent ORM

Models represent tables.

Relationships: User hasMany Websites, Website belongsTo User.

ORM allows PHP objects instead of SQL queries.

10. SSH Server Management

Connect to server:

ssh user@server-ip

cd ~/laravel_app

Useful commands:

php artisan migrate

php artisan optimize:clear

php artisan db:show

php artisan tinker

11. Password Reset

Passwords cannot be decrypted. Only reset is possible.

Reset using Tinker:

User::where('email','x@test.com')->update(['password' => Hash::make('NewPass@123')]);

12. Scheduler and Cron

Hostinger Cron Command:

php /home/USER/laravel_app/artisan schedule:run

Run every minute.

Laravel scheduler triggers WhatsApp reminder command daily.

13. WhatsApp Reminder Automation

Create command:

php artisan make:command SendExpiryReminders

Logic: Fetch expiring websites → send WhatsApp → log status.

Manual test:

php artisan billing:send-expiry-reminders

14. Data Insertion

Preferred method: Laravel Seeder or Tinker.

phpMyAdmin should be avoided for daily operations.

15. Laravel 11 Architecture

No RouteServiceProvider needed.

Routing configured in bootstrap/app.php.

Routes stored in routes/web.php and routes/auth.php.

16. Application Flow

Browser → public_html/index.php → Laravel Bootstrap → Routes → Controller → Model → DB → View → Browser

17. Security Layers

Auth middleware protects routes.

Passwords hashed using bcrypt.

Private Laravel folder prevents file exposure.

CSRF protection for forms.

18. Current Status

Login working.

Database connected.

Hosting stable.

Scheduler ready.

WhatsApp automation possible.

19. Next Development Phases

Admin Panel for managing users and websites.

User Dashboard with expiry status.

WhatsApp logs and billing reports.